

MATLAB Project: Angles

April 17, 2026

The matrix A and vector $\mathbf{b}$ are stored into MATLAB:

$$A = \begin{pmatrix} 1 & 3 & -6 & 1 \\ -1 & 1 & 5 & -6 \\ -2 & 2 & -2 & 3 \\ 4 & 0 & 5 & 2 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}.$$

MATLAB Expression	Output	Effect
<code>A * A</code>	$\begin{pmatrix} 14 & -6 & 26 & -33 \\ -36 & 8 & -29 & -4 \\ 12 & -8 & 41 & -14 \\ 2 & 22 & -24 & 23 \end{pmatrix}$	Computes $AA = A^2$, the matrix product of A with itself.
<code>A .* A</code>	$\begin{pmatrix} 1 & 9 & 36 & 1 \\ 1 & 1 & 25 & 36 \\ 4 & 4 & 4 & 9 \\ 16 & 0 & 25 & 4 \end{pmatrix}$	Computes the element-wise product of A with itself, $A \odot A$, squaring every entry of A .
<code>sum(A)</code>	$(2 \ 6 \ 2 \ 0)$	Computes the sum of the row vectors of A .
<code>sum(A .* A)</code>	$(22 \ 14 \ 90 \ 50)$	Computes a row vector whose i^{th} entry is the norm-squared of the i^{th} column of A . If $A = (\mathbf{a}_1 \ \mathbf{a}_2 \ \mathbf{a}_3 \ \mathbf{a}_4)$, then the output is $(\mathbf{a}_1 ^2 \ \mathbf{a}_2 ^2 \ \mathbf{a}_3 ^2 \ \mathbf{a}_4 ^2)$. The norm-squared represents the squared length of the vector, and it shows up in the denominator of the orthogonal projection formula.
<code>[A b]</code>	$\left(\begin{array}{cccc c} 1 & 3 & -6 & 1 & 1 \\ -1 & 1 & 5 & -6 & 5 \\ -2 & 2 & -2 & 3 & 3 \\ 4 & 0 & 5 & 2 & 2 \end{array} \right)$	Computes the augmented matrix $(A \ \mathbf{b})$ by concatenating $\mathbf{b}$ as a new column to the right of A .
<code>vertcat(A, b')</code> <code>[A; b']</code>	$\left(\begin{array}{cccc} 1 & 3 & -6 & 1 \\ -1 & 1 & 5 & -6 \\ -2 & 2 & -2 & 3 \\ 4 & 0 & 5 & 2 \\ \hline 1 & 5 & 3 & 2 \end{array} \right)$	Computes the vertically augmented matrix $\begin{pmatrix} A \\ \mathbf{b}^T \end{pmatrix}$ by concatenating $\mathbf{b}$ as a row vector to the bottom of A .

MATLAB Expression	Output	Effect
<code>R = rand(4, 6)</code>	$R = \begin{pmatrix} 0.8147 & 0.6324 & \dots & 0.6557 \\ 0.9058 & 0.0975 & \dots & 0.0357 \\ 0.1270 & 0.2785 & \dots & 0.8491 \\ 0.9134 & 0.5469 & \dots & 0.9340 \end{pmatrix}$	Generates a 4×6 matrix R whose entries are pseudorandom numbers between 0 and 1, exclusive.
<code>sortrows(A)</code>	$\begin{pmatrix} -2 & 2 & -2 & 3 \\ -1 & 1 & 5 & -6 \\ 1 & 3 & -6 & 1 \\ 4 & 0 & 5 & 2 \end{pmatrix}$	Computes a new matrix whose rows are the rows of A sorted in ascending order by their first entry.
<code>[E, index] = sortrows(A)</code> Let $\mathbf{p}$ be the index vector.	$E = \begin{pmatrix} -2 & 2 & -2 & 3 \\ -1 & 1 & 5 & -6 \\ 1 & 3 & -6 & 1 \\ 4 & 0 & 5 & 2 \end{pmatrix}, \quad \mathbf{p} = \begin{pmatrix} 3 \\ 1 \\ 4 \\ 2 \end{pmatrix}$	Computes the same sorted matrix E as <code>sortrows(A)</code> and an index vector $\mathbf{p}$ which encodes how the rows of A were rearranged to get E . The i^{th} row of E is the p_i^{th} row of A .
<code>sortrows(A, "descend")</code>	$\begin{pmatrix} 4 & 0 & 5 & 2 \\ 1 & 3 & -6 & 1 \\ -1 & 1 & 5 & -6 \\ -2 & 2 & -2 & 3 \end{pmatrix}$	Same as <code>sortrows(A)</code> , but sorts the rows of A in descending order by their first entry (instead of ascending order.)
<code>sortrows(A, 2)</code>	$\begin{pmatrix} 4 & 0 & 5 & 2 \\ -1 & 1 & 5 & -6 \\ -2 & 2 & -2 & 3 \\ 1 & 3 & -6 & 1 \end{pmatrix}$	Sorts the rows of A in ascending order by their <i>second</i> entry (instead of the first entry.)
<code>R = rand(4,6);</code> <code>S = sqrt(sum(R .* R));</code> <code>for i = 1:6</code> <code>R(:,i) = R(:,i) / S(i);</code> <code>end</code>	$R = \begin{pmatrix} 0.5795 & 0.2200 & \dots & 0.4201 \\ 0.5683 & 0.9073 & \dots & 0.3822 \\ 0.3000 & 0.0409 & \dots & 0.5543 \\ 0.5013 & 0.3559 & \dots & 0.6084 \end{pmatrix}$	Generates a 4×6 matrix R whose entries are pseudorandom numbers between 0 and 1, exclusive, and then normalizes each column of R to have length 1. The output is a matrix R whose columns are unit vectors in $\mathbb{R}^4$.

Cosine Similarity Live Script

Generates a random matrix A , then ranks each column vector of A by their cosine similarity with a reference vector $\mathbf{v}$, where:

$$\mathbf{v} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}.$$

```
A = rand(3, 10)
v = [1; 2; 3]

% Formula: cos(theta) = dot(a, v) / (norm(a) * norm(v))
norms = sqrt(sum(A .* A));           % 1. Compute norms of every column
denominators = norms * norm(v);      % 2. Compute denominators
prods = A' * v;                      % 3. Compute dot products
cosines = prods' ./ denominators;    % 4. Compute cosines

% Rank cosines in descending order
[cosines, index] = sort(cosines, "descend");
% Display ranks
for rank = 1:width(A)
    idx = index(rank);
    disp(compose("%2d. (column %d, cosine = %.4f)", rank, idx, cosines(rank)));
    disp(A(:, idx));
end
```